XianFires Accounting (ERP) System

III. Structural Diagrams (The System at Rest)

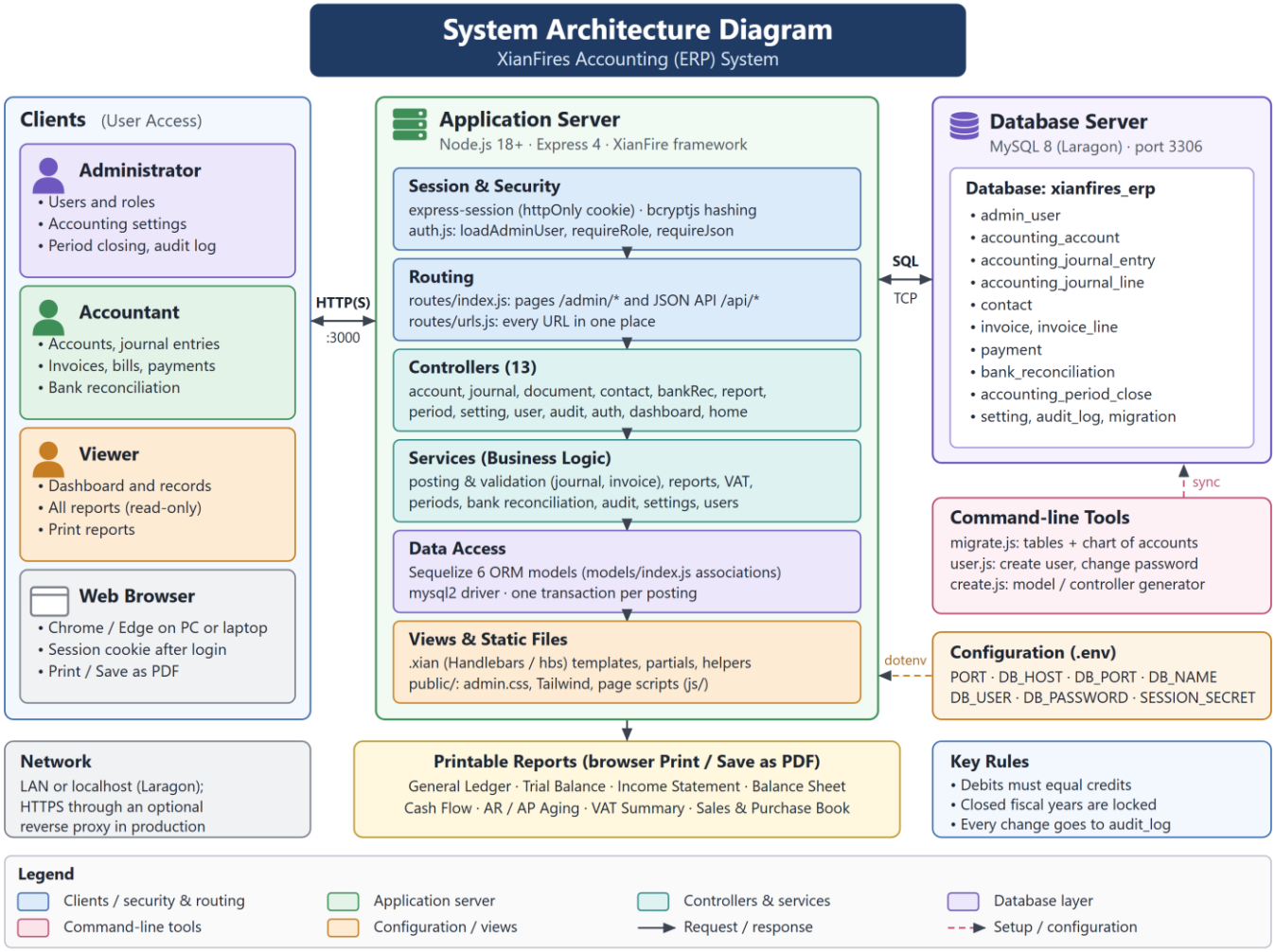


Figure 1: System Architecture Diagram

The system architecture shows how the XianFires Accounting (ERP) system reaches its users and where the bookkeeping work runs, giving stakeholders and new developers a high-level layout of the project. Administrators, Accountants and Viewers reach the system through a web browser (Chrome or Edge) on a PC or laptop over HTTP on port 3000, or over HTTPS when an optional reverse proxy is placed in front. The application server is a Node.js 18+ process built on Express 4 and the XianFire framework, layered into session and security (express-session, bcryptjs and the role middleware), routing, 13 controllers, the business-logic services, Sequelize data access and the .xian (Handlebars) views.

The MySQL 8 database server (Laragon, port 3306) holds the xianfires_erp schema: users, the chart of accounts, journal entries and lines, contacts, invoices and bills with their lines, payments, bank reconciliations, period closes, settings, the audit log and migrations. Command-line tools create the tables and the default chart of accounts (migrate.js) and manage users (user.js), while the .env file supplies the port, database credentials and session secret. Every report is printable from the browser.

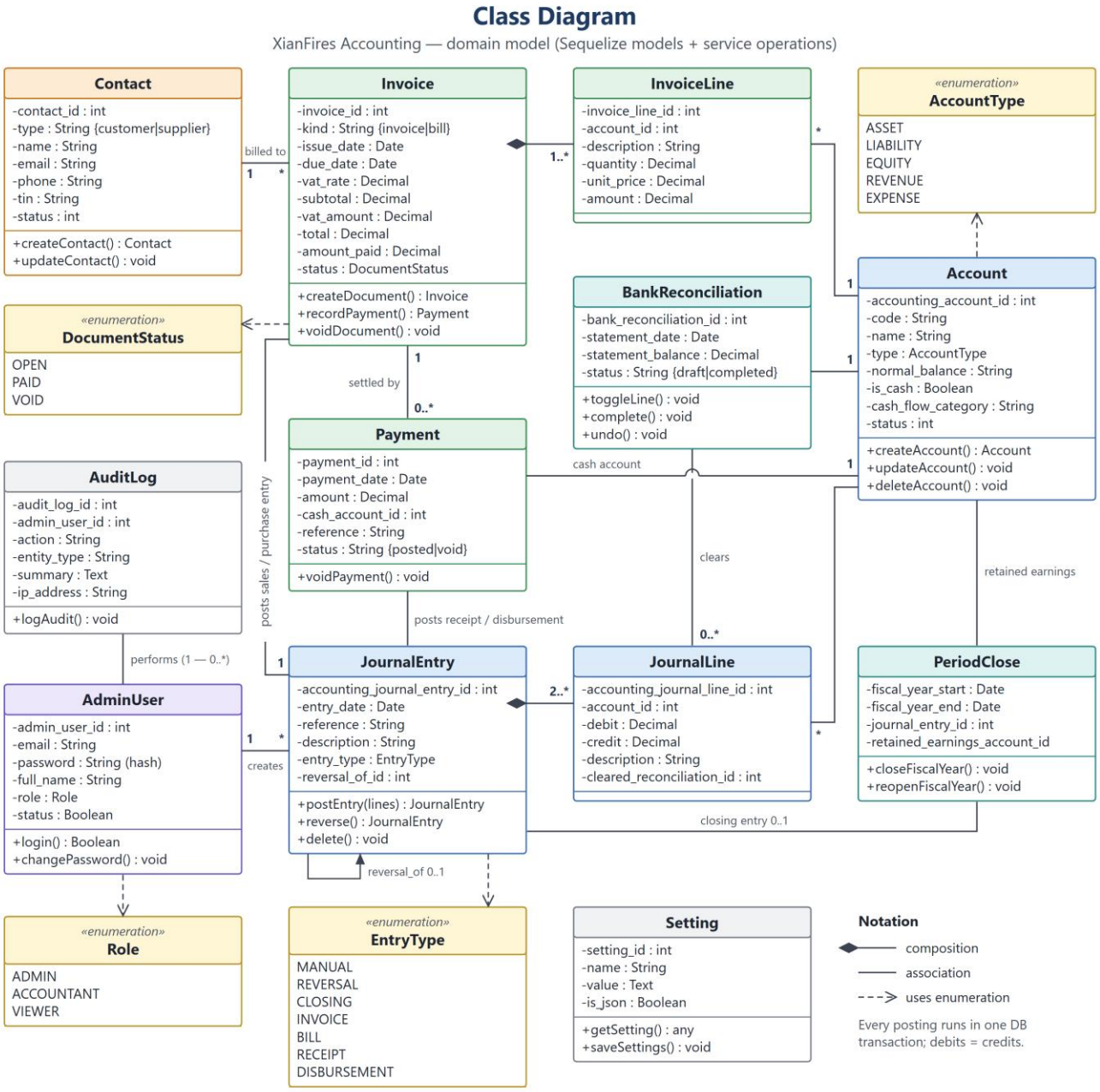


Figure 2: Class Diagram

The class structure is the back-end blueprint of the project, since each class mirrors a Sequelize model that the accounting process handles. AdminUser (role = admin, accountant or viewer) creates JournalEntry records and performs the actions kept in AuditLog. A JournalEntry is composed of two or more JournalLine objects, each pointing to one Account; its entry_type uses the EntryType enumeration (MANUAL, REVERSAL, CLOSING, INVOICE, BILL, RECEIPT, DISBURSEMENT) and reversal_of_id links a reversal to its original entry.

A Contact (customer or supplier) is billed through an Invoice, whose kind is invoice or bill and whose status uses DocumentStatus (OPEN, PAID, VOID). An Invoice is composed of InvoiceLine objects, is settled by Payment objects and posts its own journal entry, as each Payment does. BankReconciliation clears journal lines of a cash account, PeriodClose posts the closing entry to retained earnings, and Setting stores key/value configuration. The operations shown are the service functions that work on each record.

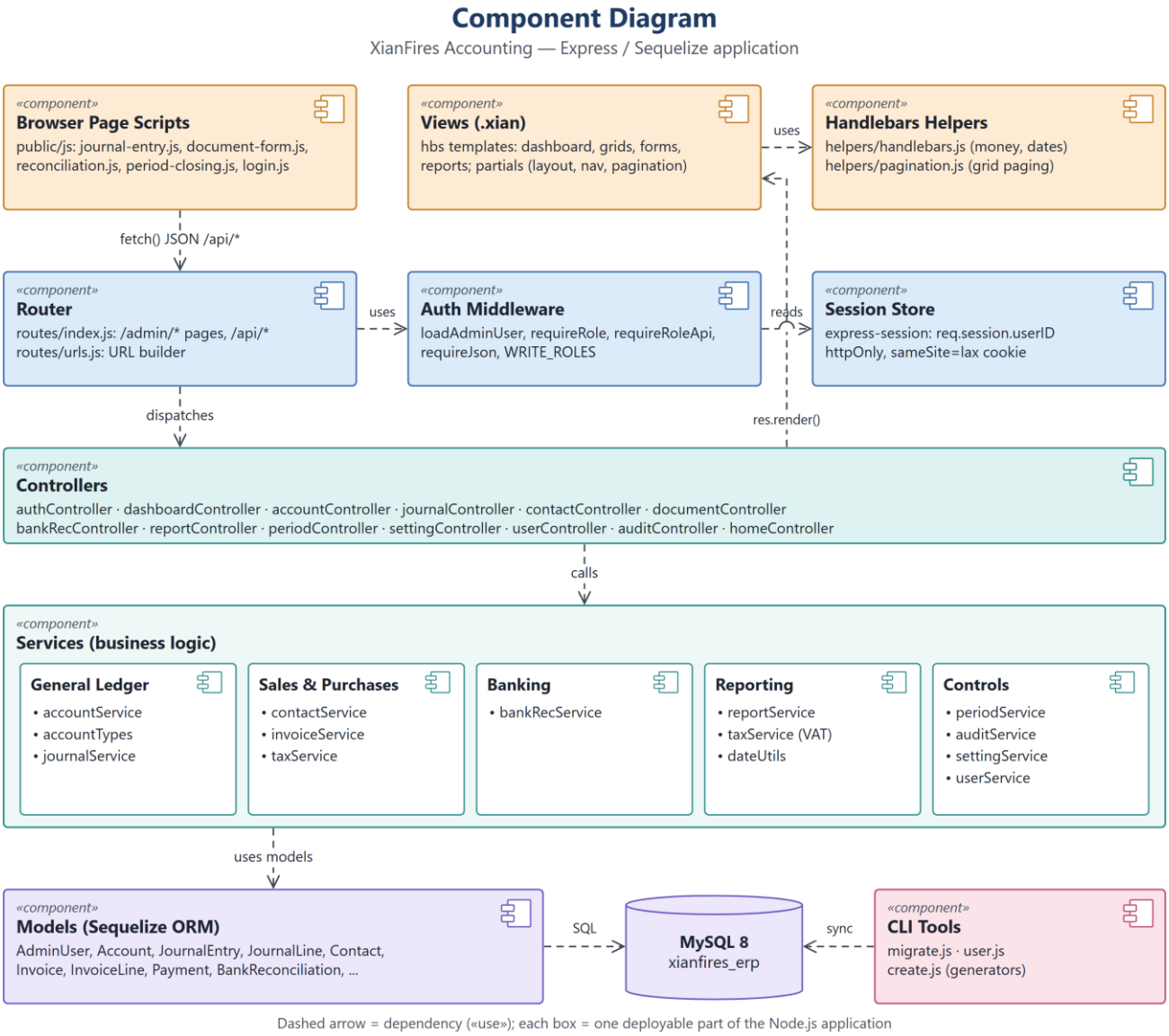


Figure 3: Component Diagram

The component structure shows how the project's Express application is assembled and how its parts depend on one another to carry out the bookkeeping. Browser page scripts (public/js) call the JSON API with fetch(), and the Router (routes/index.js and urls.js) uses the Auth Middleware, which reads the user id from the Session Store, before it dispatches the request to the Controllers. Controllers render the .xian Views, which use the Handlebars helpers for money, dates and paging.

Controllers call the Services component, grouped into General Ledger (accountService, journalService), Sales & Purchases (contactService, invoiceService, taxService), Banking (bankRecService), Reporting (reportService) and Controls (periodService, auditService, settingService, userService). The services use the Sequelize Models to reach the MySQL database, and the CLI tools use the same models to create and synchronize the schema.

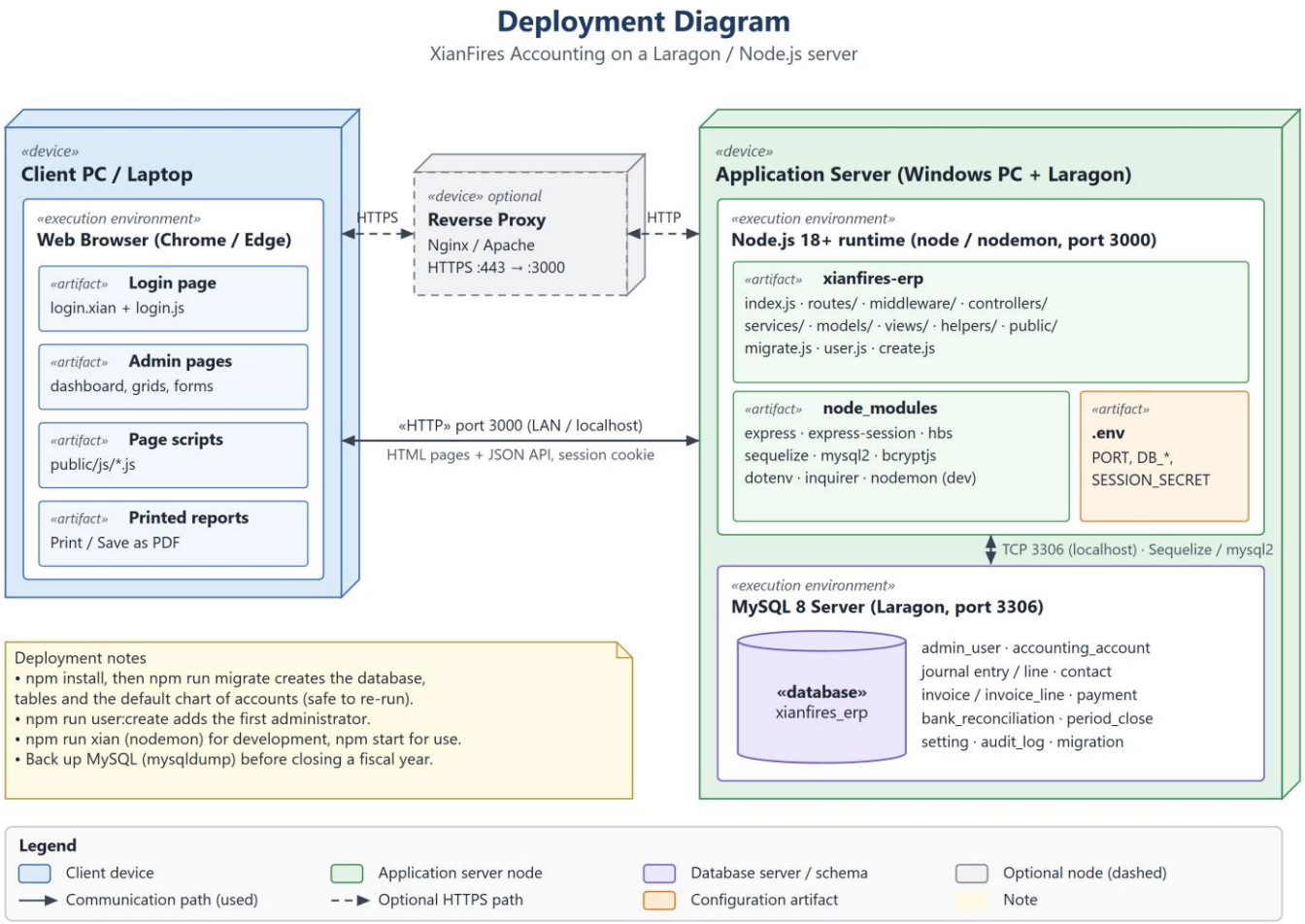


Figure 4: Deployment Diagram

The deployment setup connects the project's software to the machines that run it and guides the work needed to install the system. A client PC or laptop runs a web browser, which loads the login page, the admin pages, the page scripts and the printable reports. It talks to the application server over HTTP on port 3000 in the local network, or through an optional Nginx or Apache reverse proxy that terminates HTTPS on port 443.

The application server is a Windows PC with Laragon. Its Node.js 18+ runtime (node, or nodemon during development) runs the xianfires-erp artifact with its node_modules and .env configuration. The MySQL 8 server on the same machine hosts the xianfires_erp database and is reached through Sequelize and mysql2 over TCP port 3306 on localhost. The notes list the setup commands and a backup step before year-end closing.

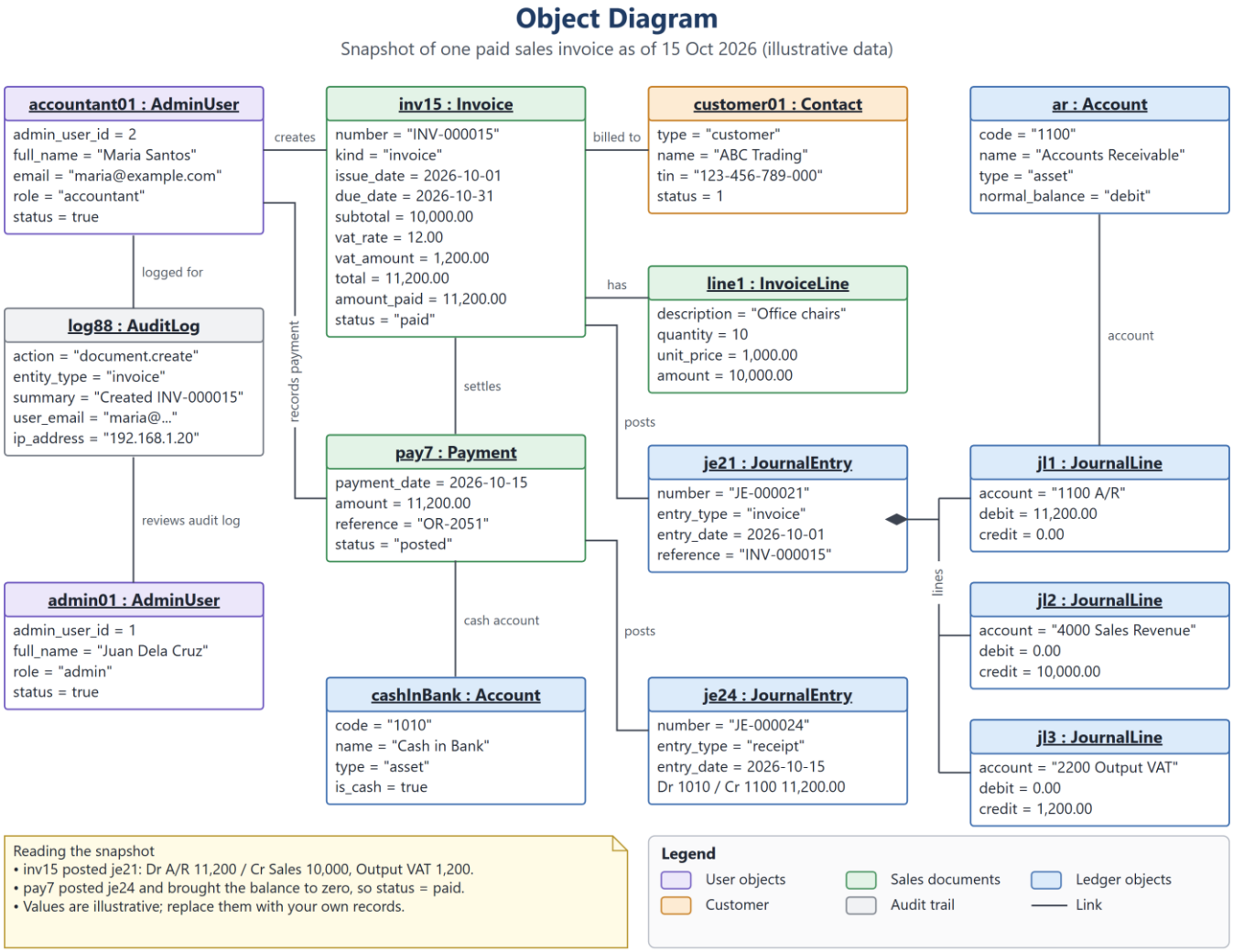


Figure 5: Object Diagram

The object snapshot ties the project's classes to real accounting data as of 15 Oct 2026, which helps verify data structures during test scenarios. accountant01 (Maria Santos, role accountant) creates inv15, sales invoice INV-000015 for customer01 (ABC Trading). The invoice has line1 (10 office chairs at 1,000.00), a subtotal of 10,000.00, 12% VAT of 1,200.00 and a total of 11,200.00, and it posts je21, whose lines debit Accounts Receivable 11,200.00 and credit Sales Revenue 10,000.00 and Output VAT 1,200.00.

accountant01 also records pay7, a payment of 11,200.00 into cashInBank (account 1010) that posts je24 (debit Cash in Bank, credit Accounts Receivable) and brings the balance to zero, so inv15 has status paid. log88 keeps the audit trail of the action and admin01 (Juan Dela Cruz) reviews it. The data is illustrative and should be replaced with actual system records.

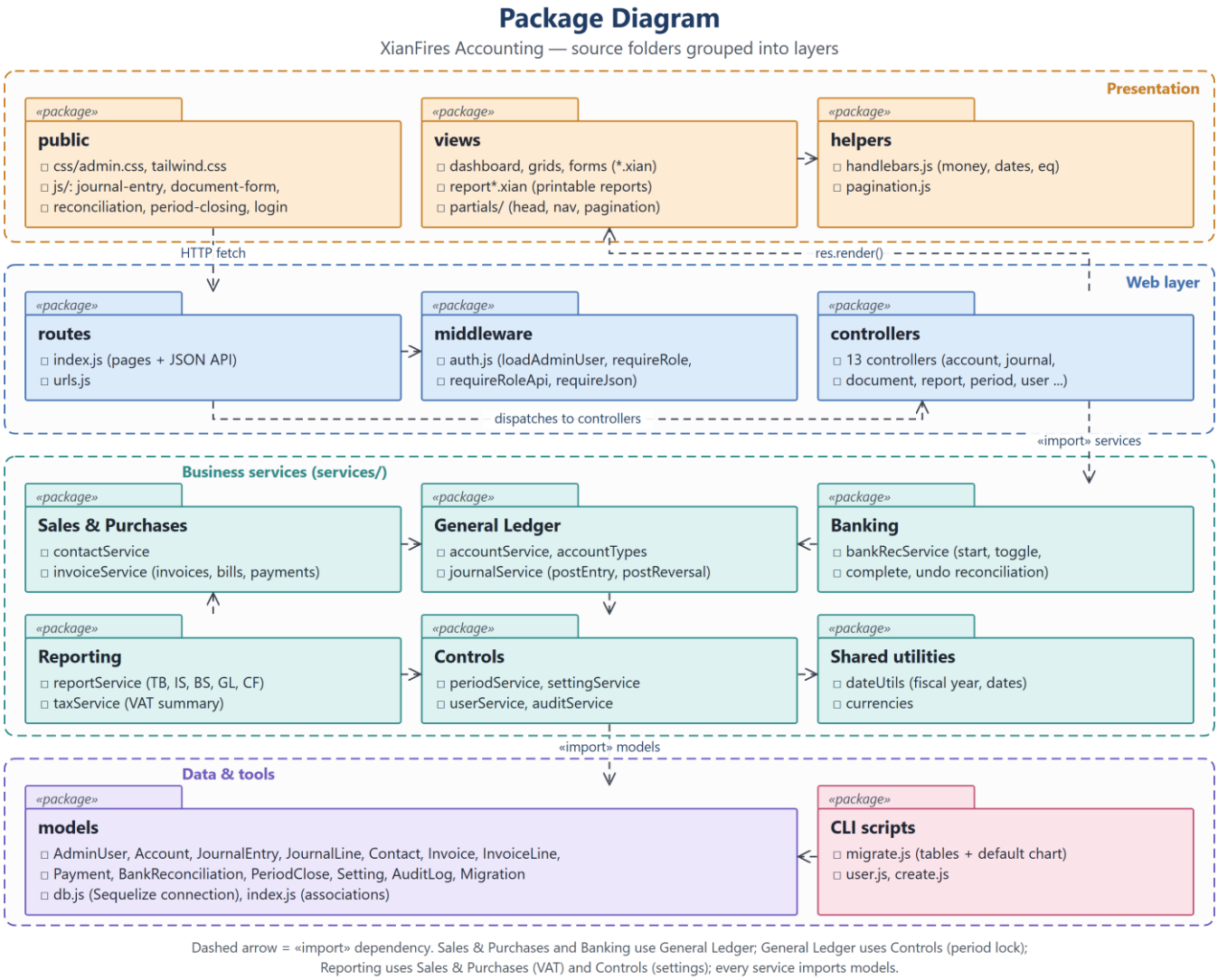


Figure 6: Package Diagram

The package structure organizes the project's source folders into layers so that presentation, web handling, accounting rules and data access stay separated and easy to maintain. The Presentation layer holds public (CSS and page scripts), views (.xian templates and partials) and helpers. The Web layer holds routes, middleware (auth.js) and controllers; routes dispatch to the controllers and the controllers render the views.

The Business services layer is split into Sales & Purchases, General Ledger, Banking, Reporting, Controls and Shared utilities. Sales & Purchases and Banking depend on General Ledger, General Ledger depends on Controls for the period lock, and Reporting depends on Sales & Purchases for VAT and on Controls for the settings. Every service imports the models package, which the CLI scripts also use.

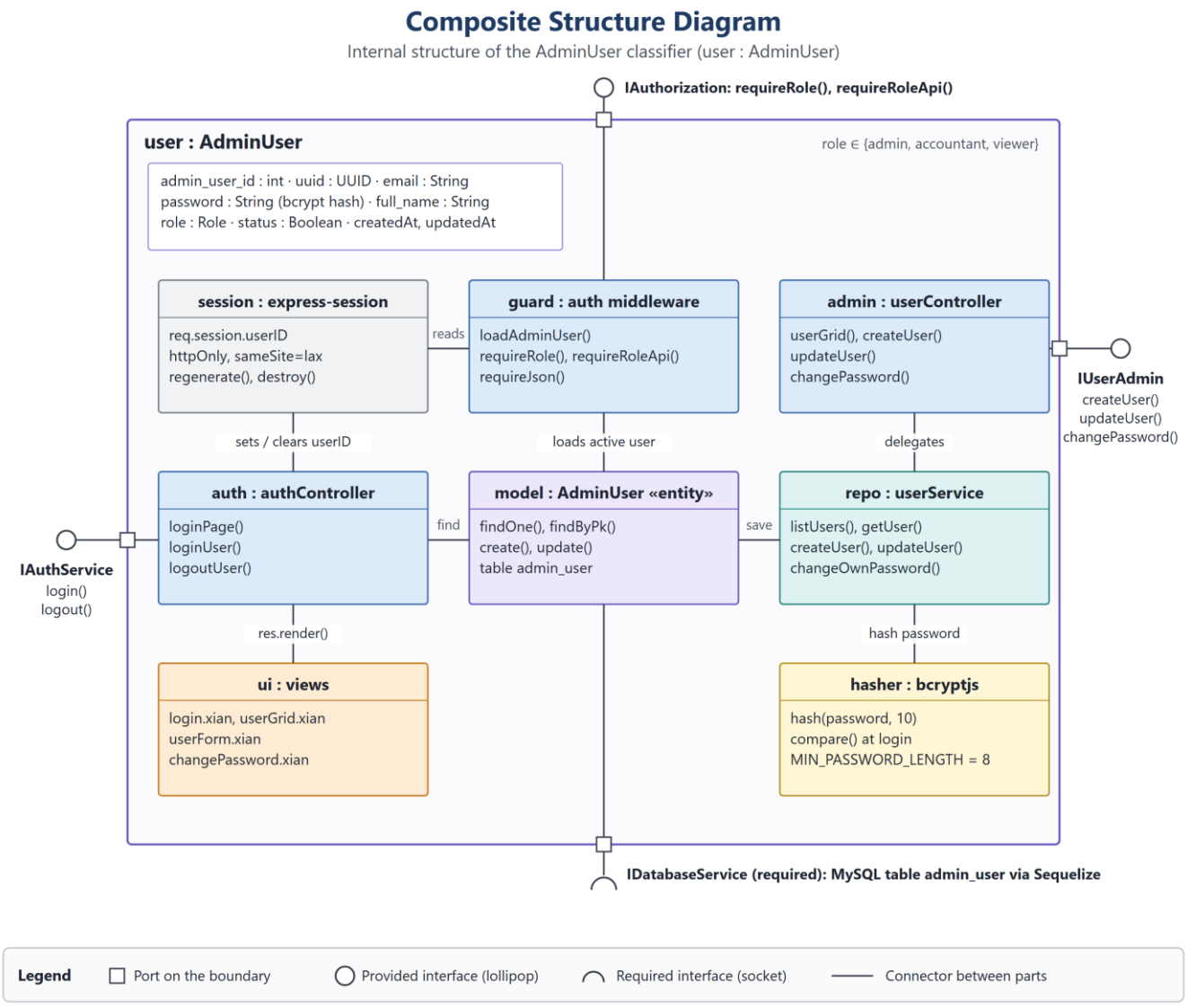


Figure 7: Composite Structure Diagram

The AdminUser class sits at the center of the project because every role and every login depends on it, so its internal structure is opened up here (user : AdminUser). It holds the attributes admin_user_id, uuid, email, password (bcrypt hash), full_name, role, status, createdAt and updatedAt. Its parts are the session (express-session), guard (auth middleware), auth (authController), admin (userController), model (AdminUser entity), repo (userService), ui (views) and hasher (bcryptjs).

Connectors show that authController sets and clears the session user id and finds the user, the middleware reads the session and loads the active user, userController delegates to userService, which saves the model and hashes passwords, and authController renders the views. The structure provides IAuthService (login, logout), IAuthorization (requireRole, requireRoleApi) and IUserAdmin (createUser, updateUser, changePassword), and requires IDatabaseService for the admin_user table.

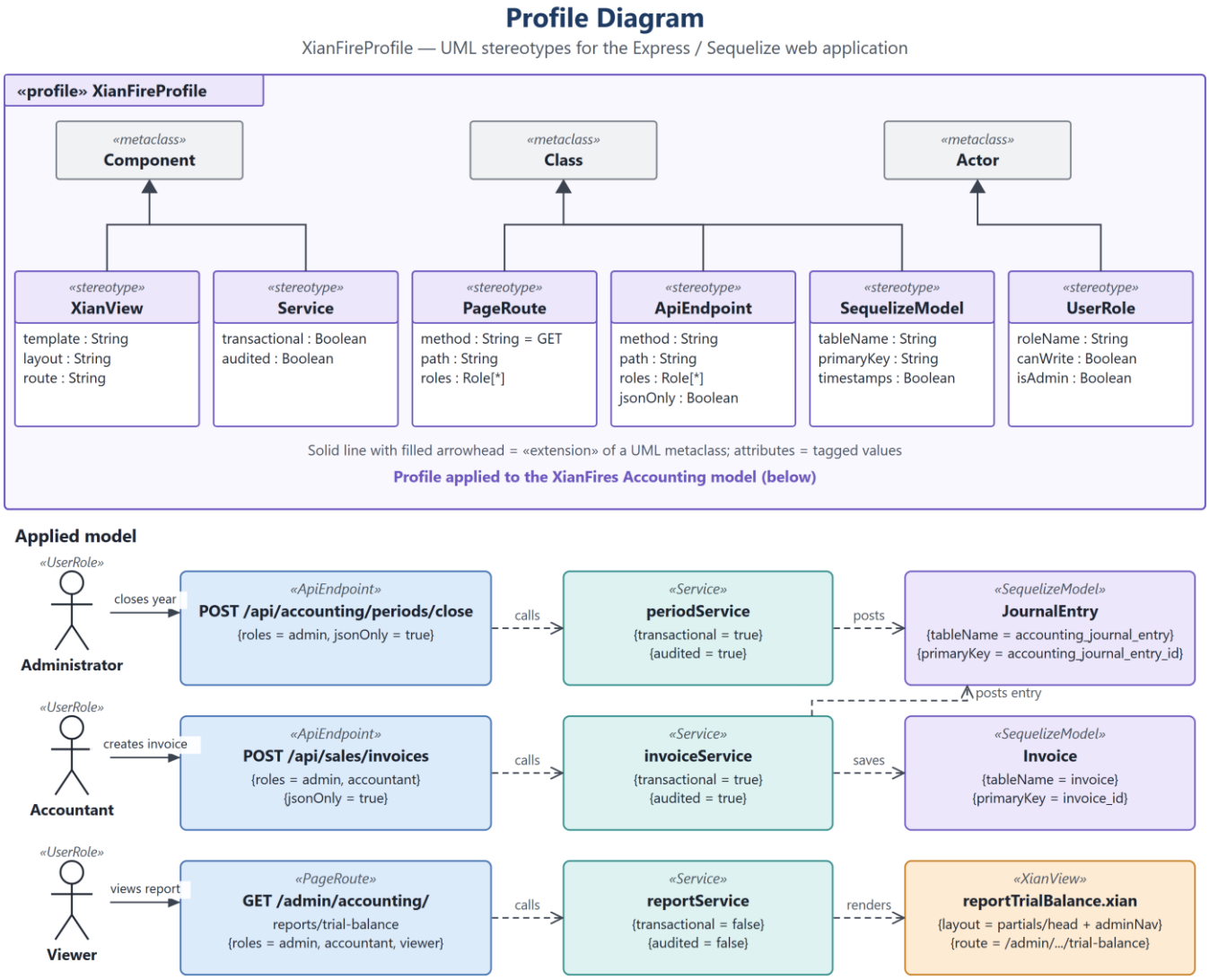


Figure 8: Profile Diagram

The XianFireProfile customizes UML for the project's Express and Sequelize web application, so its pages, API endpoints, services, database models and roles are modeled in one consistent vocabulary. It defines six stereotypes: XianView (template, layout, route) and Service (transactional, audited) extend Component; PageRoute (method, path, roles), ApiEndpoint (method, path, roles, jsonOnly) and SequelizeModel (tableName, primaryKey, timestamps) extend Class; and UserRole (roleName, canWrite, isAdmin) extends Actor.

The lower part applies them. The Administrator calls POST /api/accounting/periods/close, which calls periodService to post the closing JournalEntry. The Accountant calls POST /api/sales/invoices, which calls invoiceService to save the Invoice and post its entry. The Viewer opens GET /admin/accounting/reports/trial-balance, which calls reportService and renders reportTrialBalance.xian.

IV. Behavioral Diagrams (The System in Motion)

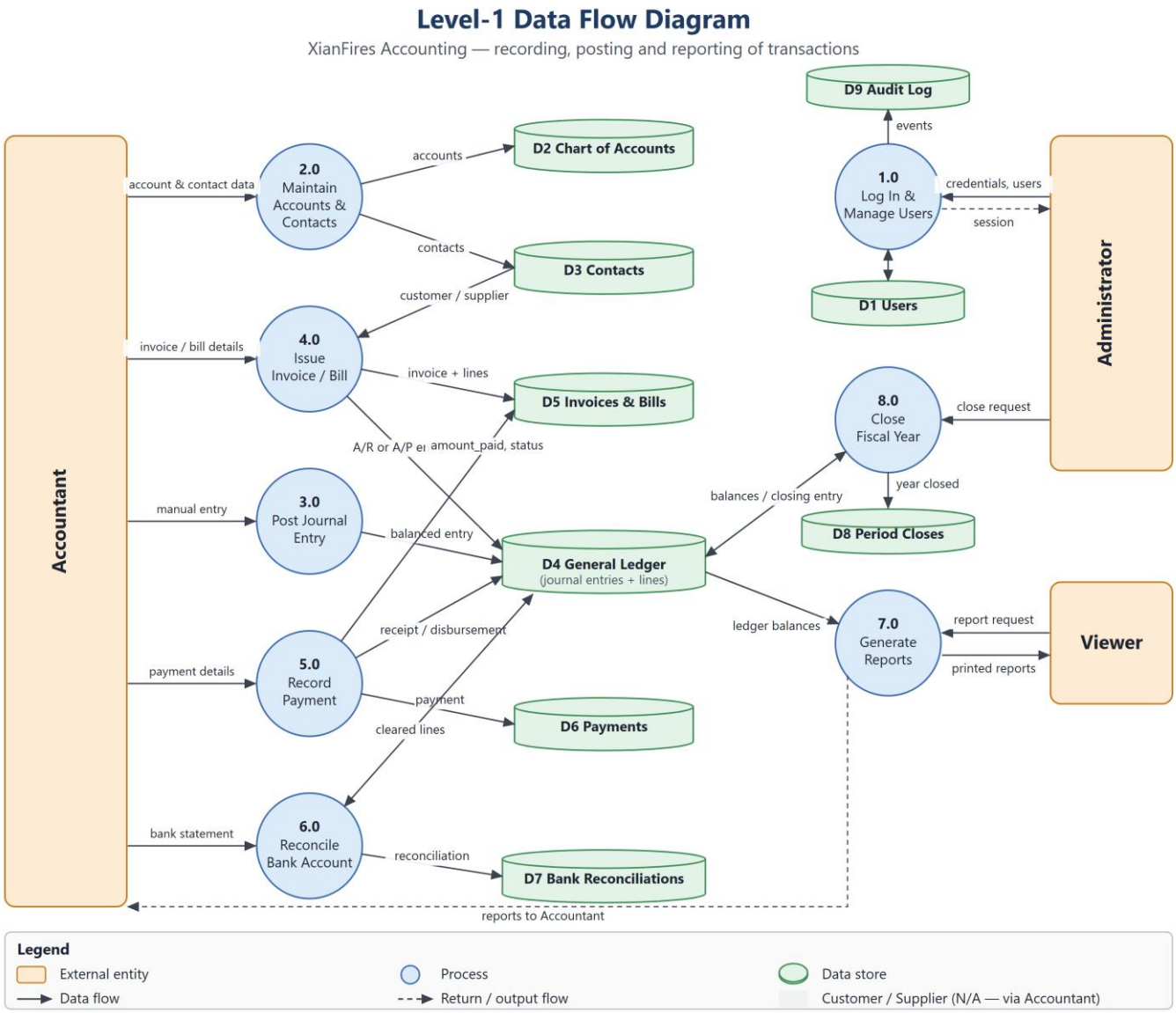


Figure 9: Data Flow Diagram (Level 1)

The Level 1 data flow traces how data moves through the project's accounting process. The external entities are the Accountant, the Administrator and the Viewer; customers and suppliers are marked N/A because their documents are entered by the Accountant. Eight processes run on nine data stores: 1.0 Log In & Manage Users uses D1 Users and D9 Audit Log; 2.0 Maintain Accounts & Contacts writes D2 Chart of Accounts and D3 Contacts; 3.0 Post Journal Entry writes D4 General Ledger.

4.0 Issue Invoice / Bill reads D3 and writes D5 Invoices & Bills and its A/R or A/P entry in D4; 5.0 Record Payment writes D6 Payments, updates amount_paid and status in D5 and posts the receipt or disbursement in D4; 6.0 Reconcile Bank Account writes D7 and marks cleared lines in D4. 7.0 Generate Reports reads ledger balances from D4 and returns printed reports to the Viewer and the Accountant, while 8.0 Close Fiscal Year posts the closing entry in D4 and records the year in D8 Period Closes.

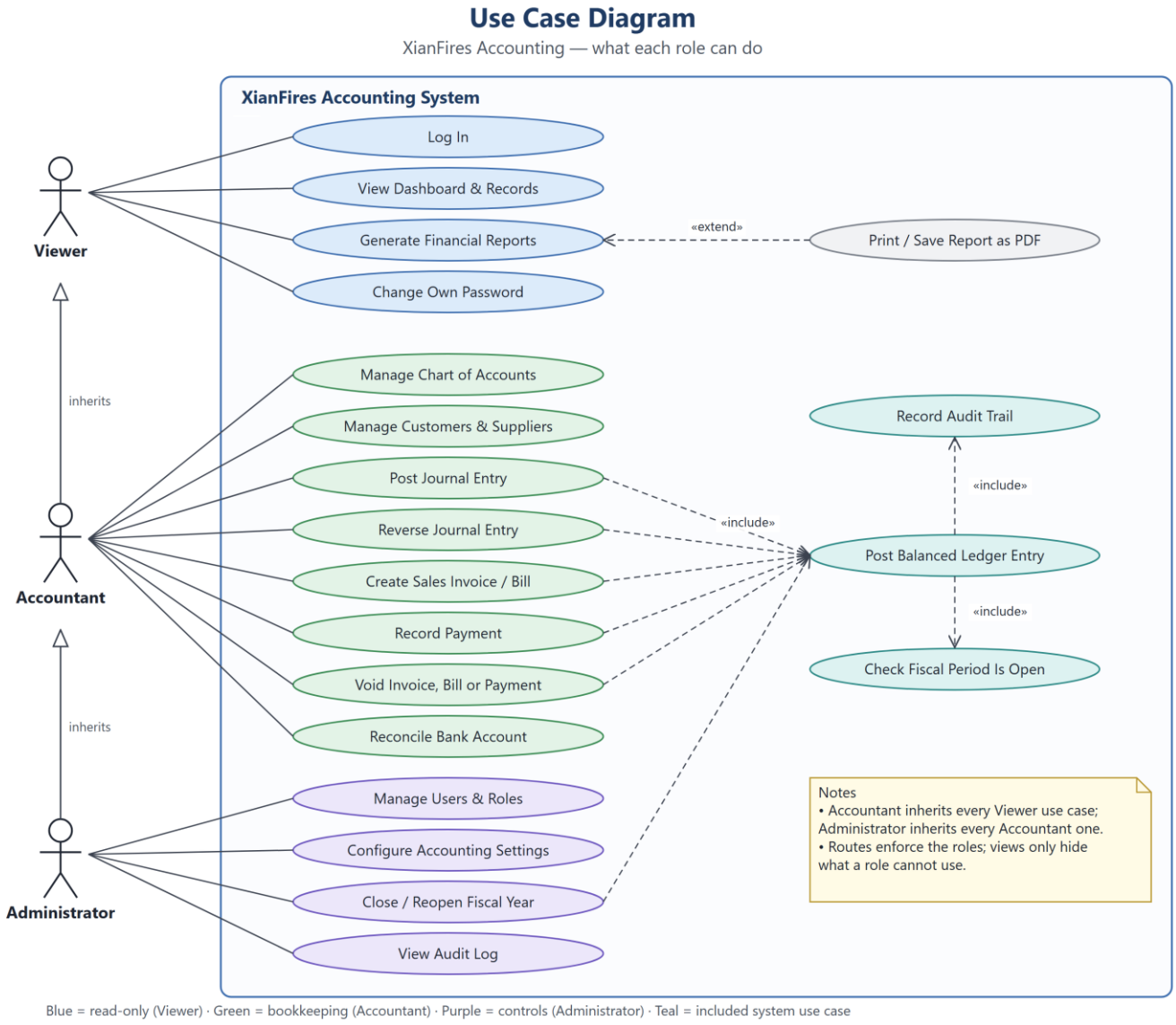


Figure 10: Use Case Diagram

The use cases define the scope of the XianFires Accounting system and show which role handles each feature. The Viewer can log in, view the dashboard and records, generate financial reports and change their own password; Print / Save Report as PDF extends report generation. The Accountant inherits every Viewer use case and can manage the chart of accounts and contacts, post and reverse journal entries, create sales invoices and bills, record payments, void documents or payments and reconcile bank accounts.

The Administrator inherits every Accountant use case and can also manage users and roles, configure the accounting settings, close or reopen a fiscal year and view the audit log. Posting, reversing, invoicing, payments, voids and year-end closing all include Post Balanced Ledger Entry, which in turn includes Check Fiscal Period Is Open and Record Audit Trail.

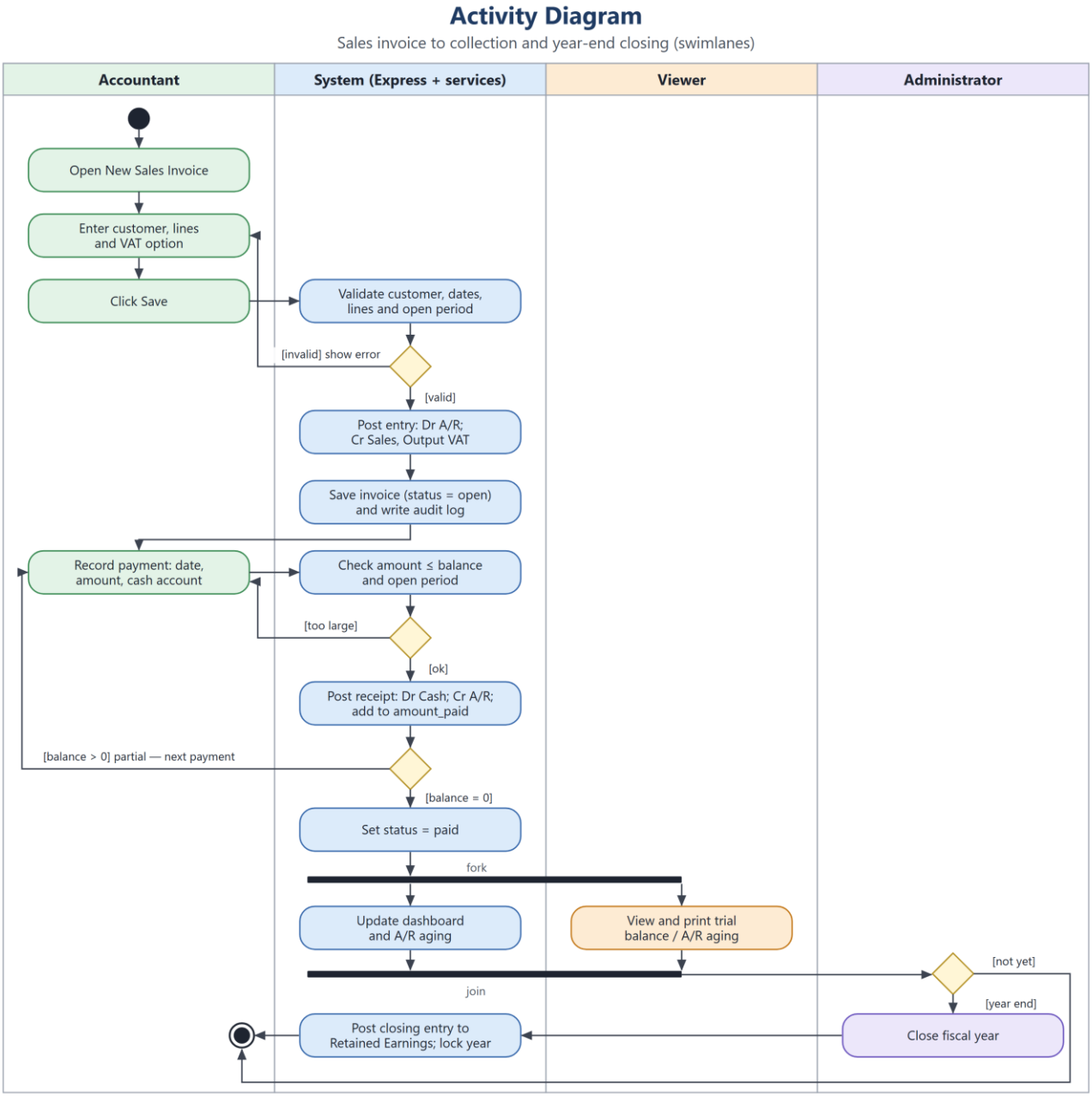


Figure 11: Activity Diagram

The swimlane workflow documents how the project handles a sales invoice from entry to collection and to year-end, across the Accountant, the System, the Viewer and the Administrator. The Accountant opens a new sales invoice, enters the customer, lines and VAT option and saves it. The System validates the customer, dates, lines and open period, returns an error for invalid data, and otherwise posts the entry (debit A/R, credit Sales and Output VAT), saves the invoice as open and writes the audit log.

The Accountant then records a payment; the System checks that the amount does not exceed the balance, posts the receipt (debit Cash, credit A/R) and adds it to amount_paid. A remaining balance loops back for the next payment; a zero balance sets the status to paid. In parallel the dashboard and A/R aging are updated while the Viewer prints the reports, and at year end the Administrator closes the fiscal year, which posts the closing entry to Retained Earnings and locks the year.

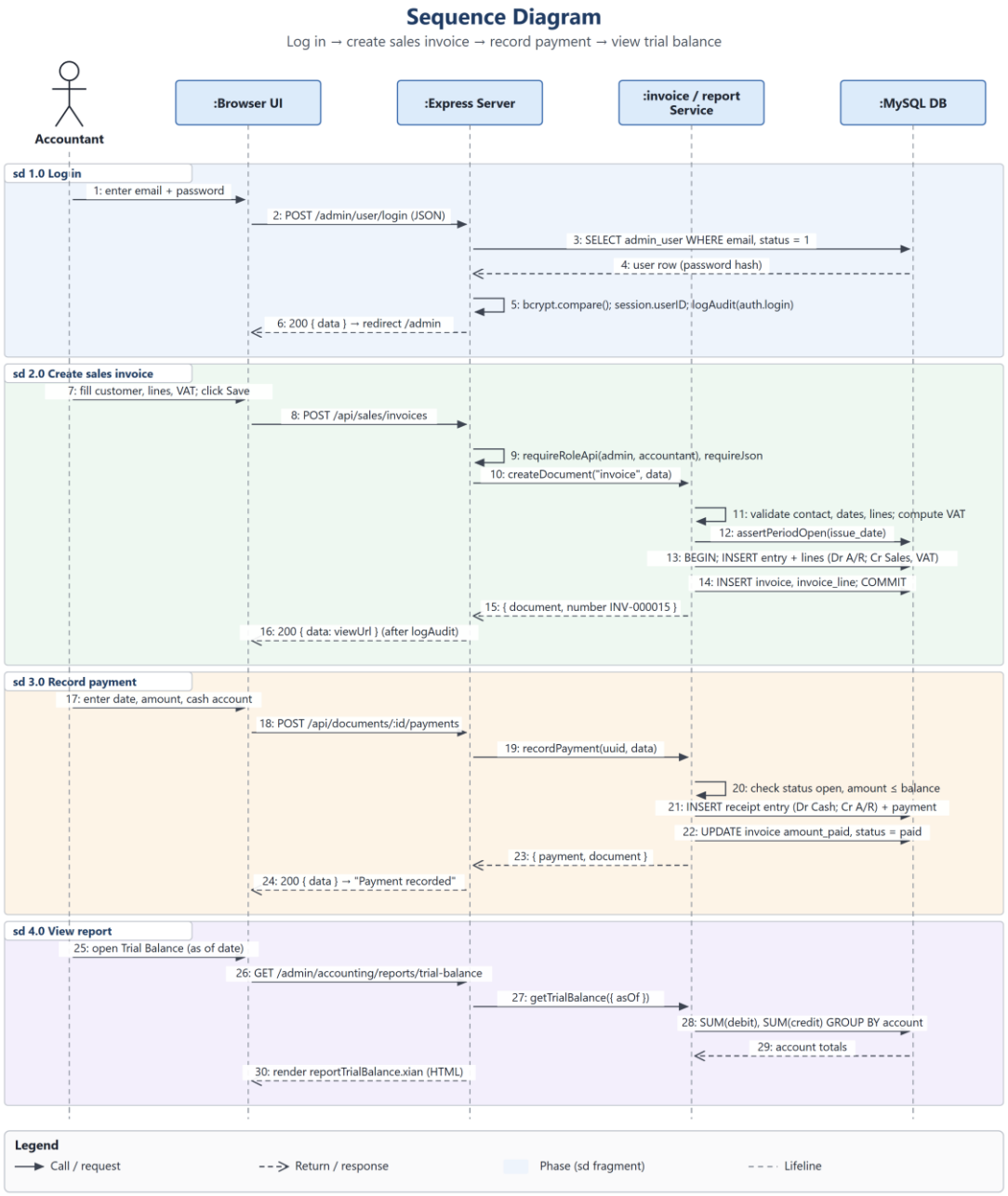


Figure 12: Sequence Diagram

The message sequence lays out how the project's main scenario runs step by step among the Accountant, the Browser UI, the Express Server, the invoice and report services and the MySQL database, using 30 messages ordered by phase: 1.0 Log in (steps 1 to 6), 2.0 Create sales invoice (7 to 16), 3.0 Record payment (17 to 24) and 4.0 View report (25 to 30).

Solid arrows are calls and dashed arrows are returns; self-calls such as bcrypt.compare(), requireRoleApi() and the validation steps run inside one lifeline. Steps 12 to 14 and 21 to 22 run inside one database transaction, so a sales invoice or payment is saved only together with its balanced journal entry.

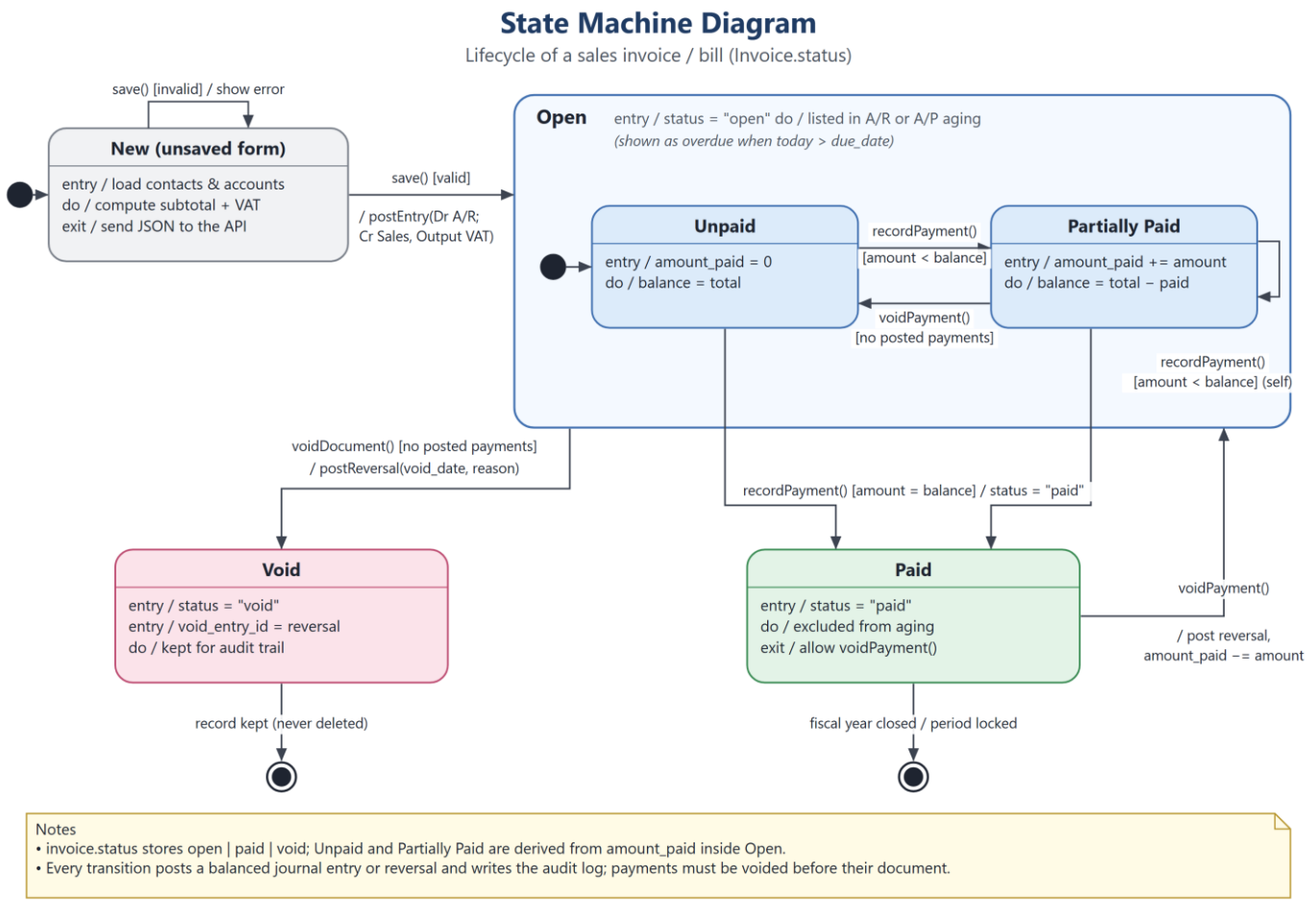


Figure 13: State Machine Diagram

The state machine follows the lifecycle of a sales invoice or bill in the project, from the moment the form is opened to its final state. It starts in New (unsaved form); save() with invalid data stays in New and shows the error, while a valid save posts the journal entry and enters Open. Inside Open the document is Unpaid, becomes Partially Paid with recordPayment() for less than the balance, and returns to Unpaid when its payments are voided.

recordPayment() for the full balance moves the document to Paid, and voidPayment() posts a reversal and returns it to Open. voidDocument(), allowed only when no posted payments remain, posts a reversing entry and moves the document to Void, which is kept for the audit trail. Each state has entry, do and exit actions, such as setting the status and listing the document in the A/R or A/P aging.

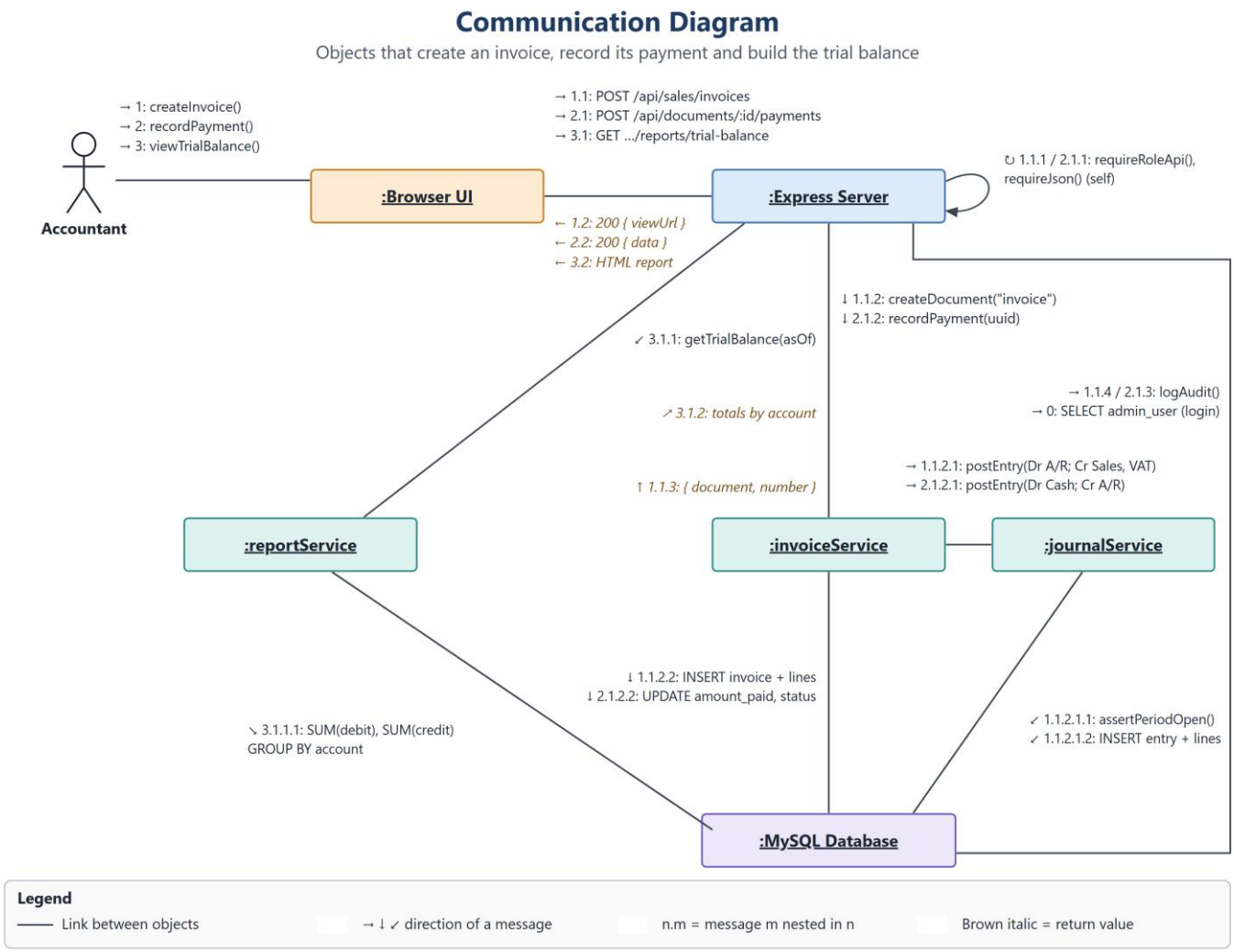


Figure 14: Communication Diagram

The communication view shows which objects work together in the project's invoice, payment and report process: the Accountant, Browser UI, Express Server, invoiceService, journalService, reportService and the MySQL database. The Accountant sends createInvoice(), recordPayment() and viewTrialBalance(); the Browser UI turns them into POST and GET requests, and the Express Server checks the role with requireRoleApi() and requireJson() before calling the services and writing the audit log.

Numbering shows order; n.m means a message nested under message n, for example 1.1.2.1 postEntry() under 1.1.2 createDocument(). invoiceService asks journalService to post balanced entries, journalService checks the open period and inserts the entry and lines, and reportService sums debits and credits per account. Brown italic messages are return values.

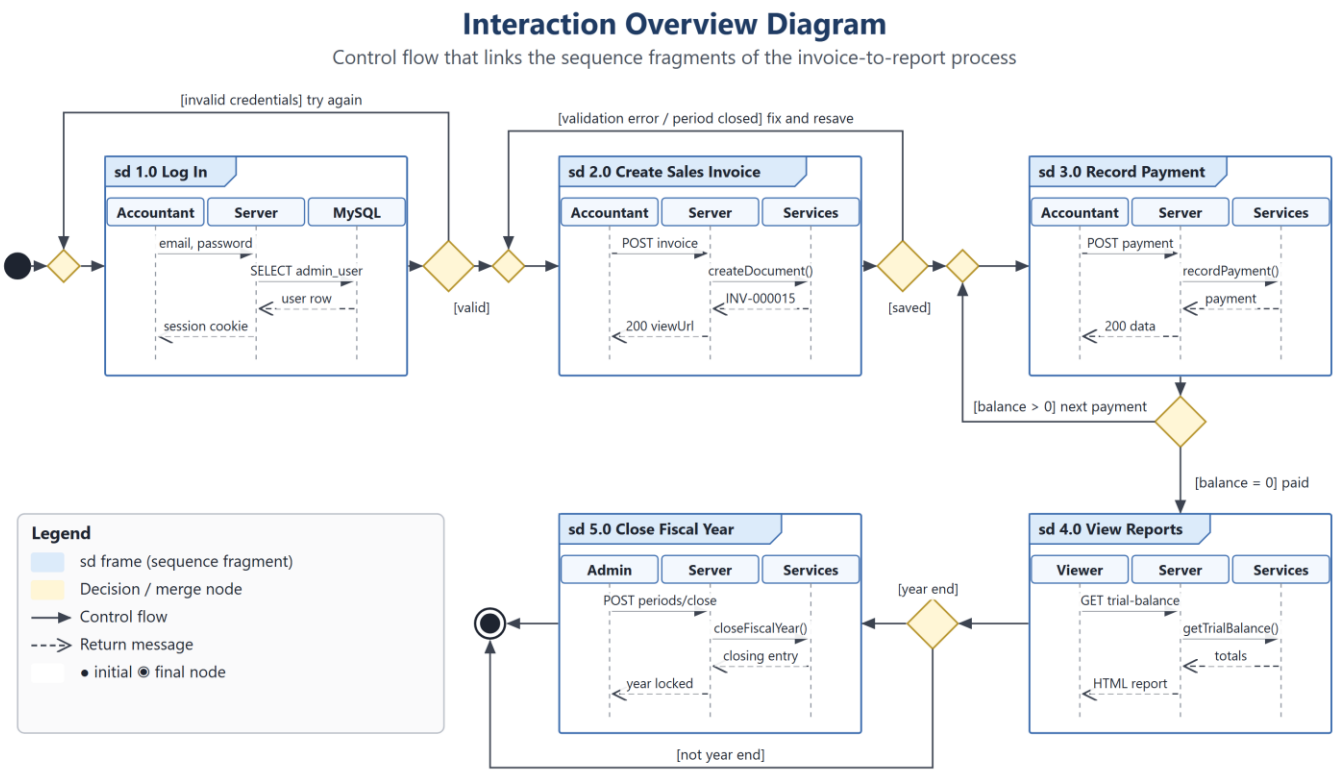


Figure 15: Interaction Overview Diagram

The interaction overview links the project's sequence frames into one control flow that covers the whole scenario: sd 1.0 Log In, sd 2.0 Create Sales Invoice and sd 3.0 Record Payment. Invalid credentials return to the log-in merge node, a validation error or closed period returns to the invoice form, and a remaining balance loops back to record the next payment.

When the balance reaches zero the flow continues to sd 4.0 View Reports. A decision then checks for the fiscal year end: if it is reached the flow runs sd 5.0 Close Fiscal Year before the final node; otherwise it goes straight to the final node.

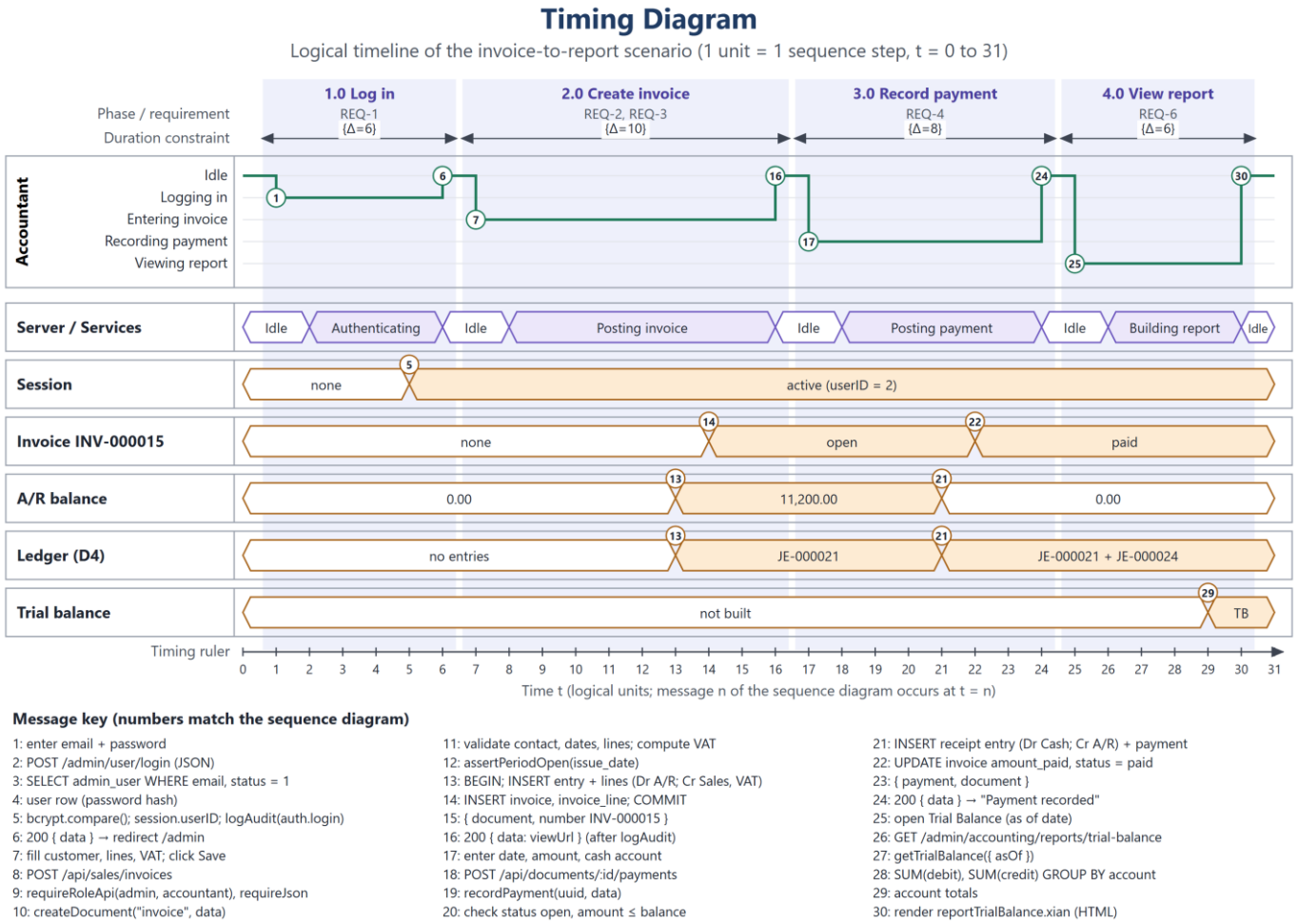


Figure 16: Timing Diagram

The timing view shows how long each stage of the project's main scenario takes and how its data changes, plotted on a logical timeline (1 unit = 1 sequence step, t = 0 to 31). Phases carry duration constraints: Log in {Δ=6}, Create invoice {Δ=10}, Record payment {Δ=8} and View report {Δ=6}, each linked to the requirement it proves.

The Accountant's state lifeline steps through Logging in, Entering invoice, Recording payment and Viewing report, while the Server / Services lifeline shows Authenticating, Posting invoice, Posting payment and Building report. The value lifelines change at the numbered messages: the session becomes active at t = 5, the ledger gains JE-000021 and A/R becomes 11,200.00 at t = 13, the invoice opens at t = 14, the payment returns A/R to 0.00 at t = 21 and the invoice becomes paid at t = 22.

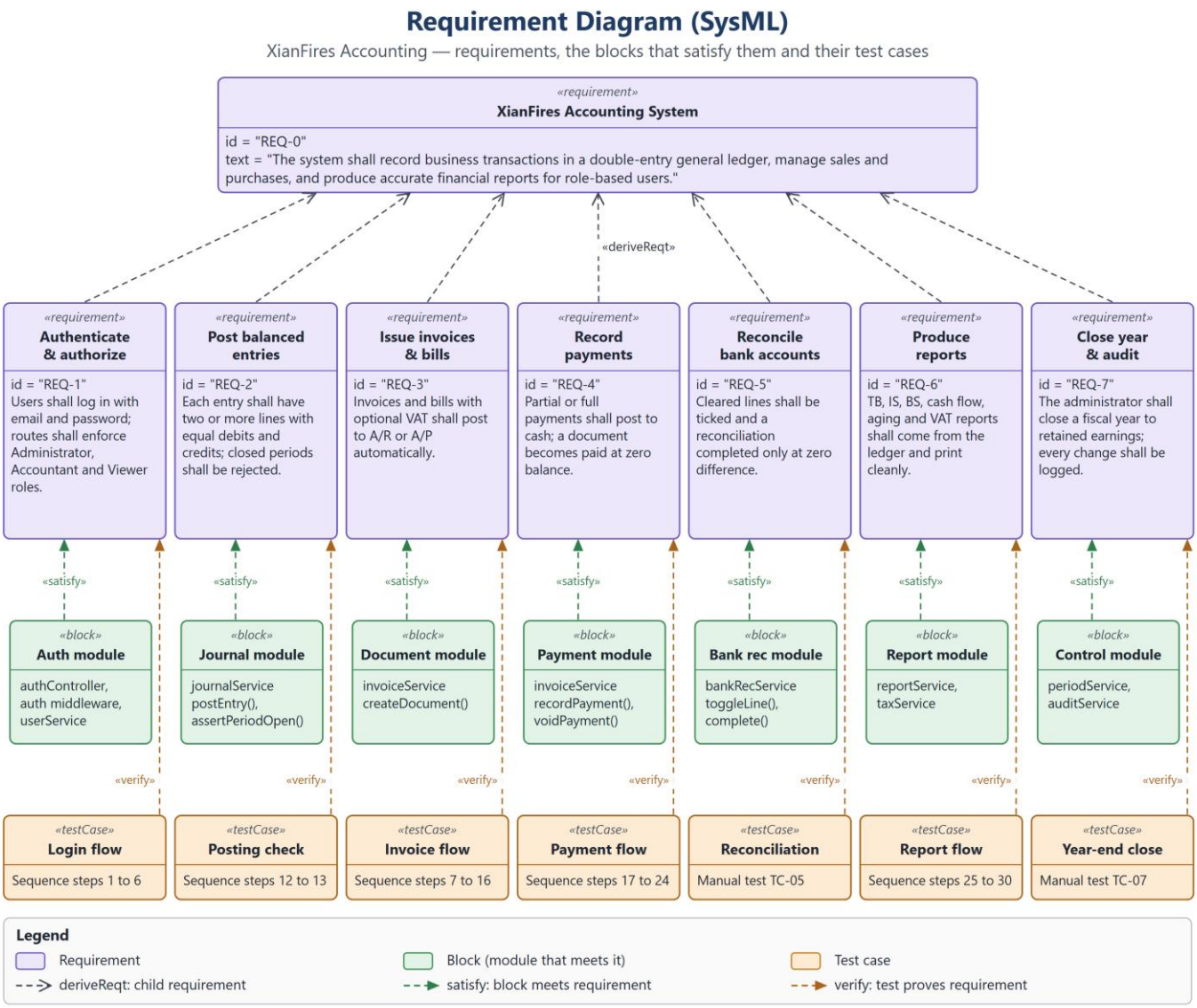


Figure 17: Requirement Diagram (SysML)

The requirements view connects the project's goals to the parts that deliver them. REQ-0 states that the system shall record business transactions in a double-entry general ledger, manage sales and purchases, and produce accurate financial reports for role-based users. It is derived into REQ-1 Authenticate & authorize, REQ-2 Post balanced entries, REQ-3 Issue invoices & bills, REQ-4 Record payments, REQ-5 Reconcile bank accounts, REQ-6 Produce reports and REQ-7 Close year & audit.

Each requirement is satisfied by a block (the auth, journal, document, payment, bank reconciliation, report and control modules) and verified by a test case taken from the sequence diagram steps, or by a manual test case (TC-05, TC-07) for the flows the sequence diagram does not cover.